Contagious Context: Self-Reproducing and Transmissible Token Strings in a Markov Cache

Dan Shumow*

June 2026

Abstract

A companion analysis [1] showed, on a toy global-plus-cache Markov model, that in-context exploitability tracks *trust*—the weight a model places on its context—and that in *generation* a single self-reinforcing injected count undergoes a *condensation* transition, locking in once trust is high enough. This paper asks what a single locked token generalizes to: can a whole *string* be made to reproduce itself, and can it *spread* between contexts? We treat the cache as the host of a self-replicating payload and make three contributions, all exact on the toy. (i) **Payload.** We characterize *quines*—strings an empty cache regenerates when run generatively. Exact reproduction obeys a window-distinctness theorem (an order-k quine of period p has p distinct k-windows and hence no proper sub-cycle), per-step fidelity follows a closed form $\rho + (1-\rho)p_{\mathrm{g}}$, and the cost to plant a contagious string splits white-box vs. black-box with a gap $a\,T\,p_{\mathrm{g}}$ that vanishes for stealthy payloads. For *approximate* quines, period *divisibility* gates a sub-cycle collapse: composite-period strings fall onto their largest proper divisor, prime-period strings are harmonic-protected. (ii) **Delivery.** Infecting a cache already warmed on a legitimate document factors as entry × lock-in; the cheapest *bridge* context balances visit frequency against single-count leverage subject to a frequency ceiling, a black-box attacker recovers the optimum from observation alone, and jointly optimizing entry and lock-in saves 3–4× over over-provisioning lock-in (the minimum sits at the condensation knee). (iii) **Transmission.** When a host that *reads* an infected output re-injects the copies it saw, one passage is a map $k_{\mathrm{in}} \to k_{\mathrm{out}}$ with a basic reproduction number $R_0 \approx T/(p\,k_{\mathrm{thresh}})$: short strings are epidemic, and there is a *critical length* above which the worm dies out after the index host. Finally, re-running all three parts under the toy's *induction* (longest-match) surrogate—its model of a transformer copy mechanism—makes self-reproducing strings several times cheaper to plant, erases the order and length penalties, and lets far longer worms sustain: contagion is most acute exactly where induction dominates.

1 Introduction

A language model carries two memories: the frozen weights and the context window. Poisoning the context window—through retrieved documents, tool output, or prior turns—is a live threat [3, 2], and a companion paper [1] studied it on a deliberately minimal surrogate: a global (parametric) Markov model linearly combined with an online Markov *cache* standing in for in-context learning. The headline there was that *exploitability tracks trust, not usefulness*, and a striking corollary

*Write-up of the analysis in the open-source project `trust-is-the-attack-surface` (branch `context-contagion`), built on the toy model `markov-models-aux-context-caching`. Every figure and number is reproduced by the `demo_contagion*.py` scripts in that repository.

appeared in generation. Because sampling a symbol increments its own count, the generative process is a nonlinear Pólya urn [7], and a single self-reinforcing injected count exhibits a *condensation* transition: shrugged off below a trust threshold, locked in above it.

That last result is about *one* token getting stuck. This paper takes it as a starting gun. If a single self-reinforcing element can capture generation, then (a) a structured *string* of tokens might be engineered to regenerate itself—a context-window analogue of *echolalia*, the compulsive repetition of what was just heard—and (b) such a string, once emitted, might be *read* by another context and reproduce there, i.e. spread. The end state is a self-replicating prompt, the toy-model cousin of the LLM "worms" demonstrated against real RAG pipelines [2]. We are after the quantitative skeleton of that phenomenon: what strings reproduce, how cheaply they can be planted in an occupied cache, and when reproduction within a host becomes an epidemic across hosts.

The reason to ask these questions of the toy is the same as in [1]: the combined predictor genuinely *is* a Markov chain, so the relevant objects—de Bruijn graph structure, hitting probabilities, Pólya condensation, an epidemic threshold—are exact rather than heuristic. We recap only what is needed (§2) and refer to [1] for the model's motivation and the trust/usefulness analysis. The three parts that follow are the payload (§3), the delivery (§4), and the transmission (§5); we then re-run all three under the toy's induction surrogate (§6), the mechanism the conjecture names.

2 Model and definitions

We use the architecture of [1]—a from-scratch analogue of the parametric-versus-in-context split studied by Bietti et al. [4]—and adopt its notation. A frozen global model g over a vocabulary of size V supplies a smoothed predictive distribution $p_{\mathrm{g}}(\cdot \mid h)$ for history h. An online order-k *cache* holds counts $N_c(\cdot)$ for each length-k context c, with total $n_c = \sum_w N_c(w)$, and the combined next-symbol distribution is the Dirichlet (Bayesian) mixture

$$P(w \mid h) \;=\; \frac{N_c(w) + a\, p_{\mathrm{g}}(w \mid h)}{n_c + a}, \qquad c = \text{last } k \text{ symbols of } h, \tag{1}$$

with prior strength a. The weight the prediction places on the cache,

$$\rho_c \;=\; \frac{n_c}{n_c + a}, \tag{2}$$

is the *reliance* (trust). Generation is prequential: the model samples $w \sim P(\cdot \mid h)$, appends it, and *updates the cache with its own sample*—the Pólya feedback responsible for condensation. Unless noted we take $V = 32$, global order 2, $a = 1$, and the two-layer synthetic source of [1] (a high-entropy shared law plus per-document structure); all numbers are reproduced by the cited scripts.

Definition 1 (Quine)**.** A cyclic string $S = (x_0, \ldots, x_{p-1})$ of period p is a *quine at order k* if, after an initially empty order-k cache observes S, seeding generation with the last k symbols of S regenerates S. We call S *exact* if the order-k context-to-successor map induced by S is single-valued, and *approximate* otherwise.

We will measure reproduction at the level of *transitions*: given S, its successor map is $\sigma(c) = x_{i+1}$ for the window c ending at x_i, and per-step fidelity is the fraction of generated steps obeying σ. Scoring positionally against a fixed phase is misleading—a single slip shifts the phase and tanks all later positions even while the cache faithfully reproduces—so transition fidelity is the honest observable.

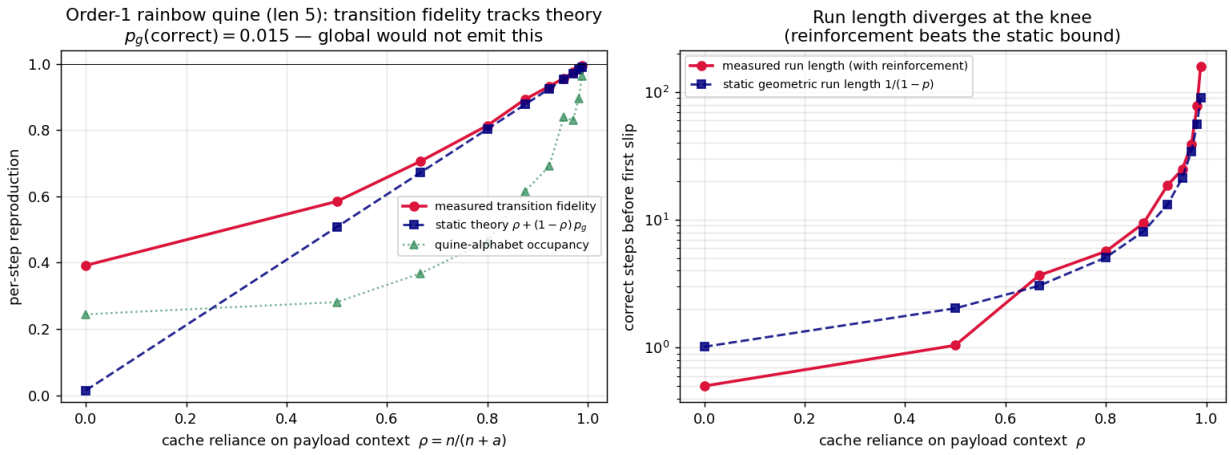

Figure 1: Order-1 rainbow quine over the rarest symbols ($p_{\mathrm{g}} = 0.015$). *Left:* measured transition fidelity tracks the closed form (3); alphabet occupancy lags but $\to 0.96$ (the walk re-enters after a slip). *Right:* measured run length beats the static geometric bound $1/(1 - p_{\mathrm{step}})$ once past the knee—self-reinforcement. Reproduced by `demo_contagion.py`.

3 The payload: self-reproducing strings

3.1 Per-step reproduction obeys a closed form

The simplest exact quine at order 1 is a *rainbow cycle* of distinct symbols $x_0 \to x_1 \to \cdots \to x_0$ (the constant run $x^*x^* \cdots$ is the degenerate length-1 case). After r copies are streamed in, each cycle context carries r counts on its unique successor, so reliance is $\rho = r/(r + a)$ and (1) gives the per-step probability of staying on the quine exactly:

$$p_{\mathrm{step}}(\rho) \;=\; \rho + (1 - \rho)\, p_{\mathrm{g}}, \tag{3}$$

where p_{g} is the global prior on the correct transition. The expected number of correct steps before the first slip is the geometric $1/(1 - p_{\mathrm{step}})$, which diverges as $\rho \to 1$: the reproduction threshold is the condensation knee of [1], now governing a whole string rather than one token.

Figure 1 tests (3) on a rainbow quine over the five globally *rarest* symbols (so $p_{\mathrm{g}} \approx 0.015$ on its transitions; the global would essentially never emit it, and all reproduction is cache trust). Measured transition fidelity tracks (3) closely—within about 0.01 across the sweep, and to three digits in the contagious high-reliance regime. It sits *slightly above* the static curve there, and across the sweep the measured run length *exceeds* the static geometric bound—e.g. 159 steps measured versus 91 predicted at $\rho = 0.989$—because each correct emission increments its own count, nudging ρ upward: the quine *self-heals*.

3.2 The order ladder and a window-distinctness theorem

The canonical exact quine at order k is a de Bruijn cycle $B(2, k)$ [6] of period 2^k over a two-symbol alphabet. Sweeping orders 2–5 (Figure 2) gives two facts. First, *per-step contagion is order-blind*: in the contagious regime every order tracks the same (3). Second, *brittleness is doubly exponential in order*: surviving one whole period needs $p_{\mathrm{step}}^{2^k}$, so the reproduced periods before a slip fall sharply with k (at $\rho = 0.996$: $63, 30, 15, 9$ periods for $k = 2, \ldots, 5$). Order is a brittleness dial, not a per-step one.

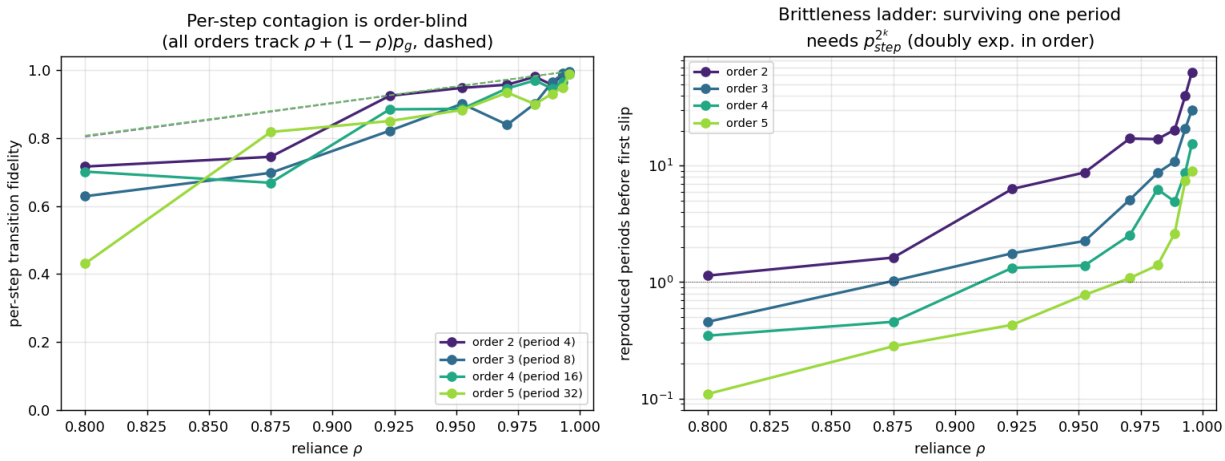

Figure 2: Order ladder, binary de Bruijn quines $B(2,k)$, $k = 2$–5. *Left:* per-step fidelity is order-blind (all track (3)). *Right:* surviving one period needs $p_{\text{step}}^{2^k}$, so reproduced periods fall steeply with order—the brittleness ladder. Reproduced by `demo_contagion_orders.py`.

The following structural fact makes the divisibility question of §3.3 well-posed.

Theorem 1 (Window distinctness)**.** *A cyclic string is an exact order-k quine of genuine period p if and only if its p length-k windows are all distinct. Consequently its de Bruijn subgraph is a single p-cycle of out-degree-one nodes, which has no proper sub-cycle.*

Proof. Exactness means each window has a single successor, i.e. each node of the order-k de Bruijn graph visited by S has out-degree one; the walk S is then a closed deterministic walk. If some window recurred, determinism would force identical continuations from both occurrences, making S periodic with a proper divisor period—contradicting genuine period p. Hence the p windows are distinct and S traces a simple p-cycle, which (all out-degrees being one) contains no shorter closed walk. The converse is immediate. □

We verified Theorem 1 on 2157 random exact quines (orders 1–4, periods 2–12, alphabets 2–4) with zero counterexamples. Its corollary is decisive: *for exact quines, period factorization is irrelevant*—there is no divisor sub-cycle to collapse into. When the generated stream settles after a slip, it returns to the full period, never to a proper divisor (confirmed across orders 2–5). Divisibility can matter only once exactness is relaxed.

3.3 Approximate quines: divisibility gates collapse

Relax exactness with a *quasi-periodic* payload: m blocks that share a period-d motif and differ only by a leading tag symbol, so the genuine period is $p = md$ but the order-1 structure is just the motif. Resolving the m blocks (hence the full period) needs cache order $\geq d$, because the disambiguating tag sits d positions back; below that order the payload collapses onto the period-d sub-cycle, a divisor of p.

Taking d to be p's largest proper divisor and running an order-1 cache (Figure 3) confirms a sharp number-theoretic prediction: composite-period payloads settle onto their largest proper divisor ($8 \to 4$, $9 \to 3$, $12 \to 6$, $15 \to 5$, $16 \to 8$, $25 \to 5$), while *prime*-period payloads $(7, 11, 13)$ have no nontrivial sub-motif and reproduce the full period. A prime period is *harmonic protection*: the

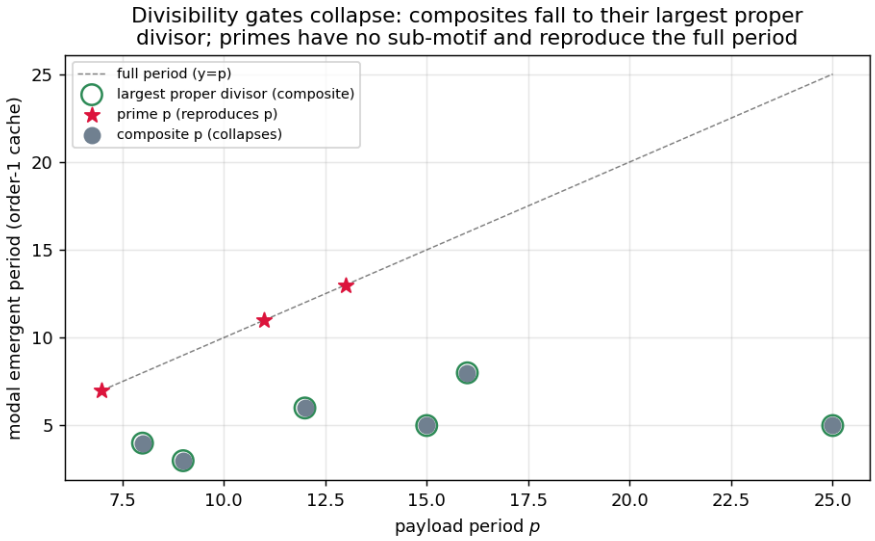

Figure 3: Approximate (quasi-periodic) quines, order-1 cache. Composite periods collapse onto their largest proper divisor (open circles); prime periods reproduce the full period (stars). Divisibility gates the available sub-cycle attractors. Reproduced by `demo_contagion_divis.py`.

string either reproduces or breaks to noise, but cannot phase-lock to a shorter cycle. Sweeping cache order at fixed composite p, the divisor-collapse fraction falls to zero exactly at $k = d$, the resolving order; but escaping the harmonic by raising order is not free—it is paid in the brittleness of §3.2, so full reproduction needs jointly $k \geq d$ and high reliance.

3.4 The cost of a payload: white-box versus black-box

How much injection plants a quine to a target per-step probability q (equivalently expected run length $T = 1/(1-q)$)? Because (3) is exact, the answer is closed-form when the global is known.

Proposition 1 (Minimal payload)**.** *For a self-loop payload with global prior p_g on its transition, the minimal repetitions to reach per-step probability q under* (1) *is*

$$r^* \;=\; a\,\frac{q - p_g}{1 - q} \;=\; a(T(1 - p_g) - 1). \tag{4}$$

Proof. Set $p_{\text{step}} = (r + ap_{\text{g}})/(r + a) \geq q$ and solve for r. □

A *white-box* attacker who knows p_{g} and a simply evaluates (4). A *black-box* attacker, with only sampled generations, has a model-agnostic fallback: the worst case $p_{\text{g}} = 0$ gives $r_0 = a(T - 1)$, which *guarantees* the target against any global because reliance dominates the prior. The value of knowing the model is therefore the gap

$$r_0 - r^* \;=\; a\,T\,p_{\text{g}}, \tag{5}$$

and it *vanishes as* $p_g \to 0$: against a maximally stealthy (rare) payload, knowing the model buys the attacker nothing (Figure 4, left). With queries, an adaptive doubling-plus-bisection search on a chosen-prefix one-step oracle recovers r^*, the error shrinking as the inverse square root of queries spent (Figure 4, right). Thus contagion is always achievable black-box, and black-box ≈ white-box exactly in the adversarial corner.

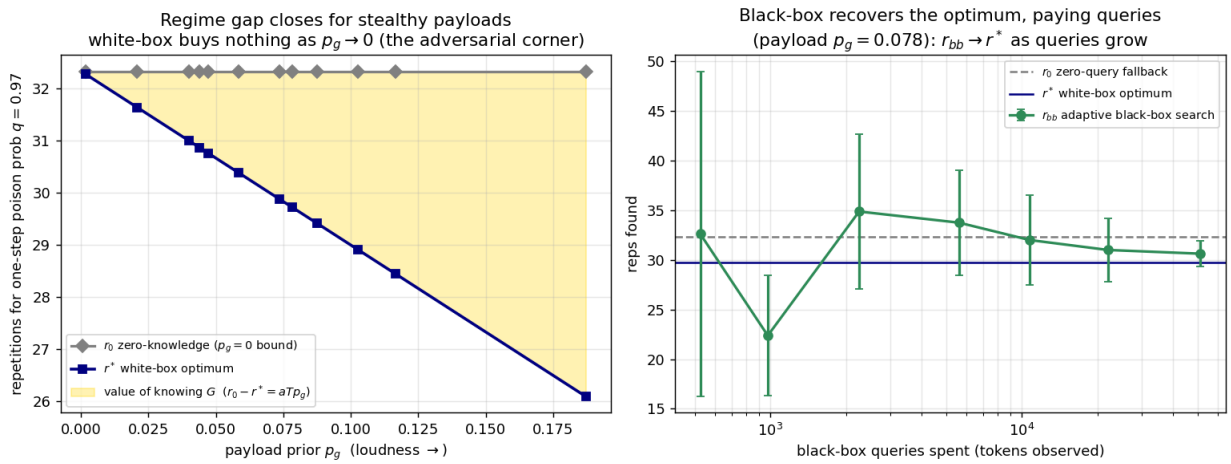

Figure 4: Planting cost by attacker knowledge. *Left:* the white-box optimum r^* and the zero-knowledge bound r_0; their gap $a\,T\,p_g$ (shaded) closes as the payload becomes stealthier. *Right:* an adaptive black-box search converges to r^* as it spends queries. Reproduced by `demo_contagion_regimes.py`.

4 The delivery: infecting a trained cache

A realistic target is not an empty cache but one already *warmed* on a legitimate document D, whose generation rides D's local structure. We take the payload token x^* to be *out-of-distribution*—a trigger the legitimate system never emits, sanitized out of the corpus and D—so its contexts are fresh and its spontaneous emission rate is zero. The hijack factors into two independent events:

$$P(\text{hijack}) \;=\; \underbrace{P(\text{enter the payload basin within } T)}_{\text{entry}} \times \underbrace{P(\text{lock-in} \mid \text{entered})}_{\text{condensation}}.$$

Lock-in is the condensation of [1]; the new object is *entry*.

4.1 Entry, the bridge, and a frequency ceiling

To enter, generation must visit a context c from which a poisoned transition routes to x^*. The attacker injects a *bridge* edge $c \to x^*$ with w counts; the added per-step entry rate is

$$\text{freq}(c) \cdot P(x^* \mid c) \;=\; \text{freq}(c) \cdot \frac{w}{n_c + w + a}, \tag{6}$$

the product of how often generation visits c and the single-count leverage there. The two factors trade off: frequent contexts are heavily warmed (high n_c, low leverage), so the best *bridge* is one both on D's trajectory and under-warmed by this particular D (Figure 5). Minimal poison to make entry likely ($P_{\text{enter}} \geq \frac{1}{2}$ over a horizon of $T = 60$ steps) is 3 counts for the best bridge max $\text{freq}(c)/(n_c + a)$, versus 8 for the most-visited context (penalized by its high n_c) and 13 for a random one. A *frequency ceiling* bounds the whole approach: since $P(x^* \mid c) \leq 1$, the entry rate saturates at $\text{freq}(c)$, so a context with $\text{freq}(c) < -\ln(1 - \text{target})/T$ can *never* reach the target however much poison is injected.

Remark 1 (Black-box entry-finding). The criterion that minimizes *poison-to-target* is $\arg\max_c (\text{freq}(c) - r)/(n_c + a)$ with r the rate ceiling (not the marginal $\arg\max_c \text{freq}(c)/(n_c + a)$; the two differ near

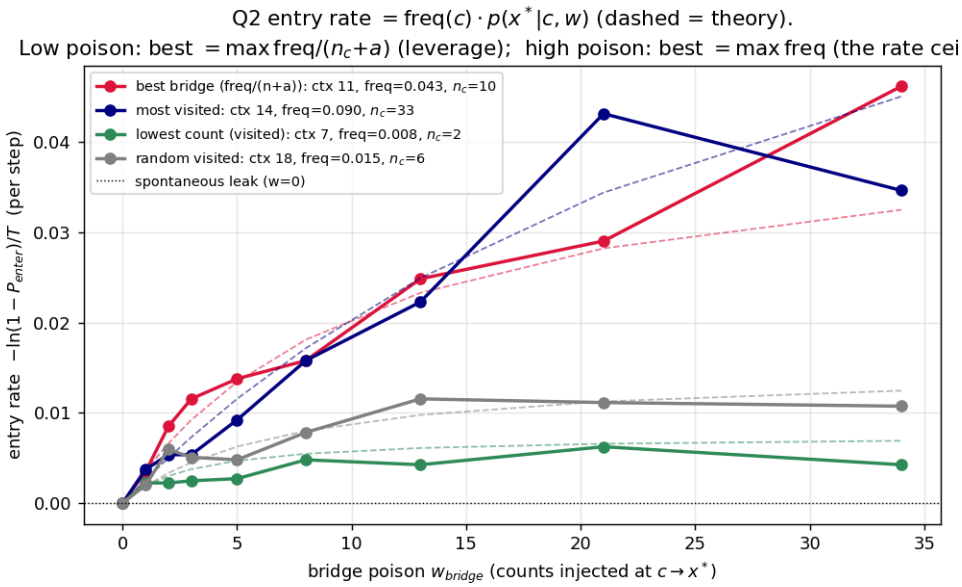

Figure 5: Entry into a cache warmed on D, payload x^* out-of-distribution (spontaneous rate 0 at $w = 0$). A bridge adds entry rate (6) (dashed: theory). Low-poison best bridge maximizes $\text{freq}/(n_c + a)$; high-poison entry rate $\to \text{freq}(c)$; low-frequency contexts saturate below target (the ceiling). Reproduced by `demo_contagion_hijack.py`.

the ceiling). For this objective visit frequency dominates: a black-box attacker that only *watches* generations recovers the white-box-optimal bridge from observation alone ($\approx$ 2000 tokens), and actively probing leverage is not worth its query cost. Defending the entry surface therefore means watching for anomalous reliance on high-visit contexts—the observable an attacker keys on.

4.2 Installing a contagious string, and the joint optimum

Bridging into a multi-symbol quine rather than a self-loop installs a contagious *string*: once entered, generation reproduces the cycle in order (measured tail cycle-fidelity 0.79–0.97 for lengths up to five), not merely visiting the payload alphabet. The total poison then splits as a length-independent entry cost plus a payload cost $r \times p$ that scales with string length.

Crucially, entry and lock-in are coupled—strong lock-in makes a single entry stick, so the bridge can be cheap; weak lock-in demands an expensive bridge—so the cheapest hijack lives on a two-dimensional frontier, not at either pinned extreme. Sweeping (w, r) and reading the minimal $w + rp$ on the $P(\text{hijack}) \geq \frac{1}{2}$ contour (Figure 6), the joint optimum sits at the condensation *knee* ($r \in [16, 32]$), not at saturation, and beats the "pin lock-in strong" recipe by 3–4$\times$ (e.g. 48 versus 136 counts at $p = 1$; 192 versus 648 at $p = 5$). Over-provisioning lock-in is wasted poison.

5 Transmission: a context-window worm

Self-reproduction within one cache becomes *contagion* when an infected output infects a second host. The mechanism makes transmission a corollary of §3. An infected host generates output in which the string S appears k times—its *viral load*. A second host that *reads* that output streams those k copies into its own cache, which is exactly the payload injection of Proposition 1 with $r = k$,

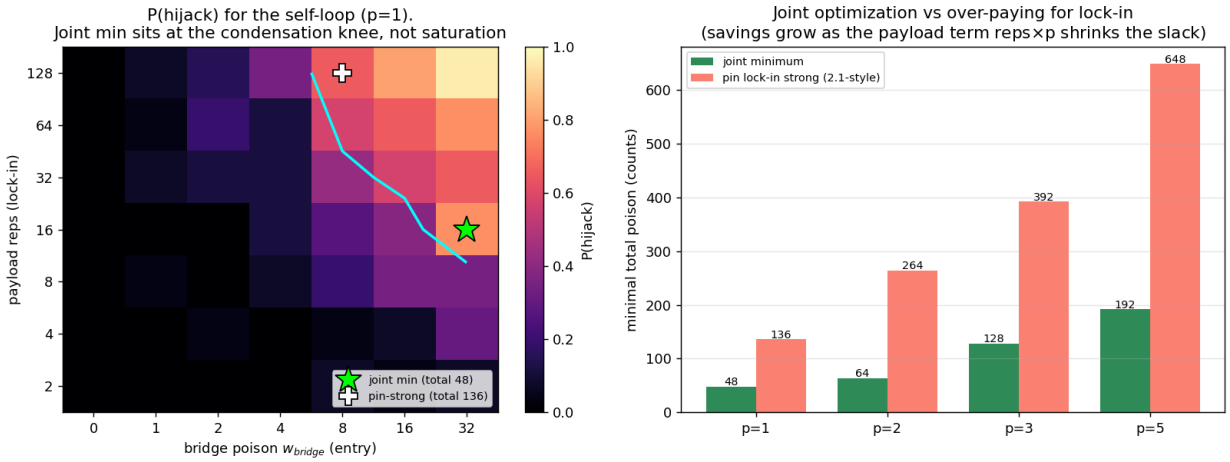

Figure 6: Joint minimal poison. *Left:* $P(\text{hijack})$ over (entry w, lock-in r) for a self-loop; the $\frac{1}{2}$ contour (cyan) is L-shaped and the joint minimum (star) sits at the knee, far below the pin-strong recipe (cross). *Right:* joint minimum versus over-provisioned lock-in across string lengths. Reproduced by `demo_contagion_joint.py`.

and the read context ends mid-string, so the receiver starts already inside the basin: *secondary infection needs no bridge*. The receiver then reproduces S and re-broadcasts k' copies in its own T-token output. One passage is a map $k_{\text{in}} \mapsto k_{\text{out}}$, and the epidemic is its iteration.

Two regimes follow (Figure 7). Below a transmissibility threshold the receiver does not reproduce and $k_{\text{out}} \approx 0$ (the chain dies). Above it the receiver reproduces and broadcasts a load $k_{\text{out}} \approx (\text{occupancy}) \cdot T/p$, a ceiling set by how much it generates (T) divided by the string length (p). The basic reproduction number [8] is the ratio of broadcast to threshold,

$$R_0 \;\approx\; \frac{\text{broadcast ceiling}}{\text{threshold}} \;\approx\; \frac{T}{p\,k_{\text{thresh}}}, \tag{7}$$

so contagiousness falls with string length and rises with how much each host emits. Serial passage from a strong index host bears this out: a length-1 string climbs to an endemic load ($\sim$90 copies/host), length 2 sustains near threshold, but lengths ≥ 3 die out within a few passages. There is a *critical string length* (here $\approx$ 2–3, scaling with T) above which $R_0 < 1$ and the worm infects only the index host. Length trades payload richness against transmissibility: the worm wants to be short.

6 Under induction: the conjecture's mechanism in the toy

Everything so far used a fixed-order count cache, whose trust is the reliance $n/(n+a)$ of (2)—earned by accumulating counts. The conjecture (§7) names a *different* trust mechanism: induction, where a span is trusted because it *matches* a previous span, not because copying it helps. The toy ships exactly this as an alternative cache. `PpmPredictor` is a variable-order online predictor that interpolates the longest previously-seen suffix down to the global model with absolute discounting—"saw $A\,B\ldots$ now see A, predict B," an induction head built from counts [5]. It is never told its order; it discovers the effective order per context. Swapping it in for the count cache tests the conjecture's mechanism within the toy, where it stays exact.

The mechanistic contrast is sharp: the count cache needs many presentations to drive reliance past the condensation knee, whereas induction earns near-full trust from a single long, near-deterministic

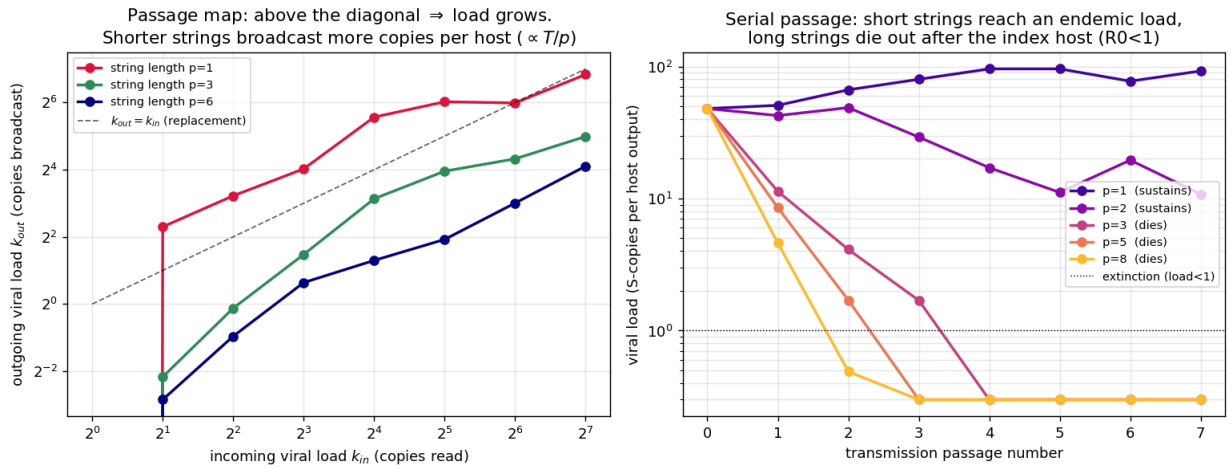

Figure 7: Cross-cache propagation. *Left:* the passage map $k_{\text{in}} \to k_{\text{out}}$; above the replacement diagonal the load grows ($p = 1$, supercritical), below it the load decays ($p = 3, 6$). *Right:* serial passage—short strings reach an endemic load, long strings die out after the index host ($R_0 < 1$). Reproduced by `demo_contagion_worm.py`.

match. Three consequences follow (Figures 8, 9).

Cheaper to plant. A rainbow quine reaches fidelity 0.9 after about three presentations under induction, versus 12–32 for the count cache, and locks sharply to 1 by the fourth (Figure 8, left). At a *single* presentation induction is in fact worse: a once-seen suffix keeps only $1 - d$ of its mass under absolute discounting, so induction must have seen a span at least twice before it copies it confidently—a small but telling threshold.

Structural reach. The brittleness ladder of §3.2 is an artifact of fixed order. For a de Bruijn $B(2, 3)$, which needs order 3, the order-1 cache cannot reproduce it at all and an order-matched order-3 cache is brittle (fidelity 0.76 at eight presentations); induction reaches 0.9 by the third presentation and 1 by the sixth, never told the order (Figure 8, right). It copies long-range structure a fixed low-order count mechanism cannot represent.

More transmissible. Re-running the worm of §5 with induction hosts collapses the transmissibility threshold. Because reproduction now needs only two or three copies, k_{thresh} in (7) falls accordingly and R_0 stays above one for far longer strings: in serial passage every length we tried (1–8) sustains, each reaching the full T/p broadcast ceiling, where the count cache died for every length $p \geq 3$ (Figure 9). The critical length jumps—long worms that count-trust kills are viable under induction.

The reading is that induction is not incidental to the conjecture but its engine: it is exactly where self-reproducing strings are cheapest to plant, where the order and length penalties vanish, and where transmission sustains longest. This is established on the toy, with the same predictor the project uses as its induction surrogate. It does not by itself settle the real-transformer question, but it removes the most obvious way the conjecture could have failed: that the cheap, length-robust contagion seen above was a peculiarity of fixed-order *counting* rather than of trust-by-*matching*.

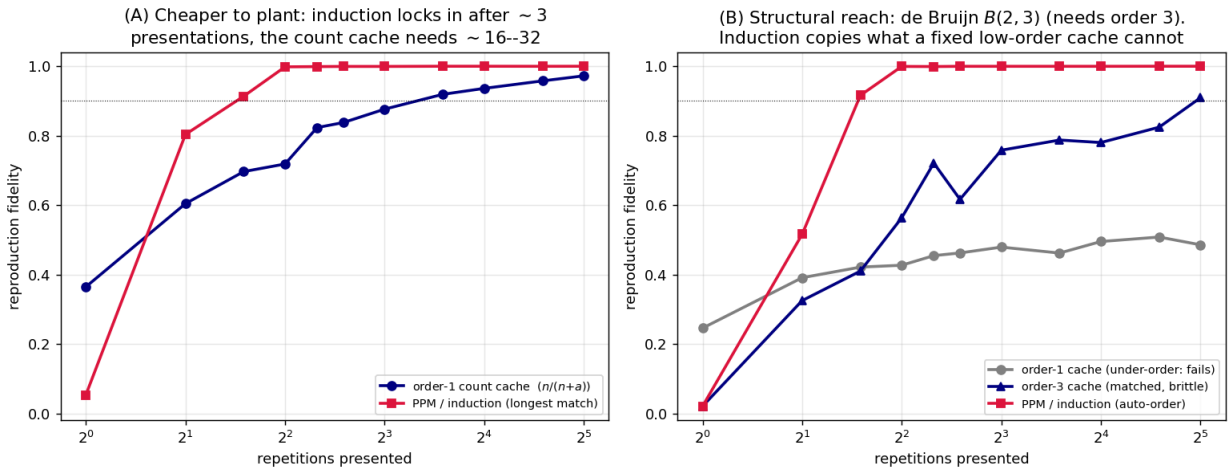

Figure 8: Contagion under the induction surrogate (PPM) versus the fixed-order count cache. *Left:* a rainbow quine reproduces after ~ 3 presentations under induction but ~ 16–32 under counting. *Right:* for a de Bruijn $B(2,3)$ (needs order 3) the order-1 cache fails and the order-3 cache is brittle, while induction—never told the order—reproduces it. Reproduced by `demo_contagion_ppm.py`.

7 Discussion

Three observations recur across the parts and are the load-bearing take-aways.

A single quantity threads the program. Reliance (2)—trust—sets per-step fidelity (3), the lock-in probability, the payload cost (4), and the transmission threshold (7). The condensation knee of [1] is where strings begin to reproduce (§3), where the cheapest hijack sits (§4.2), and where a worm becomes supercritical (§5). The paper is in this sense a single statement about trust, told at three scales.

Black-box $\approx$ white-box where it matters. Twice, independently, the information an attacker lacks turns out not to help in the adversarial regime: knowing the global model buys nothing against a stealthy payload (5), and knowing the victim's warmed counts buys nothing for entry-finding once visit frequency is observed (Remark 1). The practically dangerous attacks are the cheap, observation-only ones.

Short and simple is contagious; long and rich is not. Every length effect points the same way. Higher-order, longer strings are doubly-exponentially more brittle to reproduce (§3.2), cost $\propto p$ to plant (§4.2), and have $R_0 \propto 1/p$ to transmit (§5). A self-replicating context payload is under pressure to be a short, low-order motif—which is also the regime a defender can most plausibly enumerate and watch for.

Toward real transformers, and defenses. As in [1] the bridge to real models is a conjecture, but a sharper one here, and §6 has already supplied its toy-level evidence. The induction (copy) mechanism [5] is precisely a longest-match cache, so the de Bruijn determinism of §3.2 and the divisor-collapse of §3.3 predict that highly templated or repetitive contexts—where induction earns trust on grounds orthogonal to usefulness—are where short self-reproducing spans are cheapest to

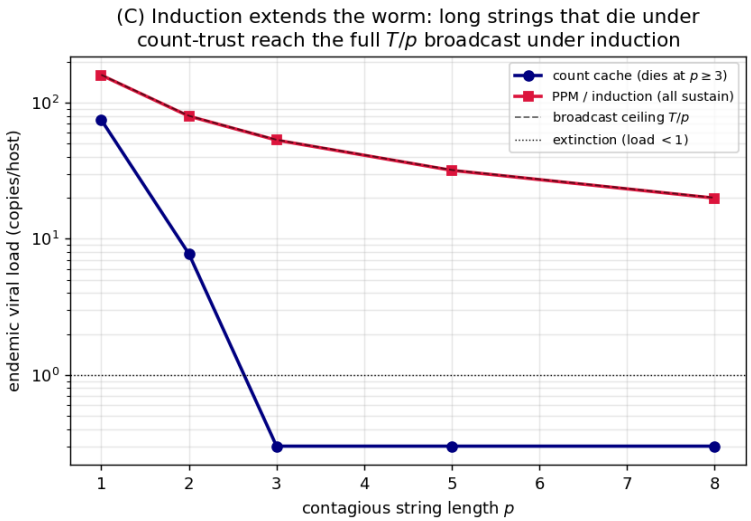

Figure 9: The worm under induction. Serial-passage endemic load versus string length: the count cache dies for $p \geq 3$, but under induction every length sustains at the full T/p broadcast ceiling—the critical length jumps because k_{thresh} collapses. Reproduced by `demo_contagion_ppm.py`.

plant and most transmissible, with prime-length (aperiodic) motifs the most robustly self-locking. The defensive reading inverts the obvious instinct in a now-concrete way: the quantity to bound is *trust placed on high-visit, repetitive context*, and the cheapest worms are short, so a watch-list of self-similar low-order spans, plus a cap on how much reliance any recurring span can earn, attacks exactly the regime the cost analysis says an attacker will use.

8 What is claimed, and what is not

Everything is established *on the toy*, where the relevant objects are exact: Theorem 1 is a graph fact (and was checked computationally), (3), (4), (5), (6) are algebra on (1), and the condensation and epidemic thresholds are standard reinforced-process and branching phenomena read off a literal Markov chain. The constructed correspondences—"payload / delivery / worm" and the epidemiological R_0—are organizing devices, not theorems about transformers. The induction results of §6 use the project's longest-match predictor (PPM); they show the phenomena are a property of trust-by-matching rather than of fixed-order counting, but PPM is a model of an induction head, not a transformer. The divisibility result is exact for the quasi-periodic construction but should not be over-read: it says period factorization gates *which* sub-cycles exist, not that natural strings are prime-protected. The joint-poison minima are grid-resolution upper bounds; the 3–4× saving and the "minimum at the knee" structure are the robust claims. The transformer conjecture of §7 is stated to be attacked, not assumed. What we do claim firmly is the qualitative skeleton: self-reproducing context strings exist and are characterized by window-distinctness and reliance; they can be planted in an occupied cache for a poison budget that is cheapest at the condensation knee and is no harder black-box than white-box where it matters; and their cross-host spread has a reproduction number that falls with length, with a critical length above which contagion does not sustain.

References

[1] D. Shumow. *Trust Is the Attack Surface: A Cryptanalytic Reading of In-Context Memory Poisoning in a Markov Mixture Model.* Companion write-up, project `trust-is-the-attack-surface`, 2026.

[2] S. Cohen, R. Bitton, B. Nassi. *Here Comes the AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications.* 2024.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.* AISec, 2023.

[4] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jégou, L. Bottou. *Birth of a Transformer: A Memory Viewpoint.* NeurIPS, 2023.

[5] C. Olsson et al. *In-context Learning and Induction Heads.* Transformer Circuits, 2022.

[6] N. G. de Bruijn. *A Combinatorial Problem.* Proc. Koninklijke Nederlandse Akademie van Wetenschappen, 1946.

[7] R. Pemantle. *A Survey of Random Processes with Reinforcement.* Probability Surveys, 2007.

[8] R. M. Anderson, R. M. May. *Infectious Diseases of Humans: Dynamics and Control.* Oxford University Press, 1991.